

PyTorch: Make Moons Classification

From a weak linear classifier to a non-linear neural network

Dataset	make_moons(n_samples=1000, noise=0.2, random_state=42)
Task	Binary classification: class 0 vs class 1
First model	Linear(2, 1) + Sigmoid
Improved model	2 $\rightarrow$ 16 $\rightarrow$ 16 $\rightarrow$ 1 with ReLU hidden layers
Final loss in uploaded notebook	0.0013 at epoch 2000
Final test accuracy in uploaded notebook	99.0%

Goal of this guide: learn the PyTorch workflow by seeing exactly what the model receives, what it outputs, how loss is calculated, how backpropagation produces gradients, and how the optimizer changes the parameters.

This guide is based on the uploaded notebook *pytorch3_classification.ipynb*. The notebook first uses a linear classifier and then a non-linear network; its final model uses two 16-neuron hidden layers with ReLU, BCEWithLogitsLoss, Adam, 2000 epochs, and reports 99.0% test accuracy.

1. The PyTorch Workflow

1. Get / create the dataset
2. Separate X (inputs) and y (labels)
3. Convert to PyTorch tensors
4. Split into train and test sets
5. Build the model
6. Forward pass: `prediction = model(X_train)`
7. Calculate loss
8. `loss.backward()` → calculate gradients
9. `optimizer.step()` → update weights and biases
10. `optimizer.zero_grad()` → clear old gradients
11. Repeat the training steps for many epochs
12. Evaluate on `X_test`

The training loop to memorize:

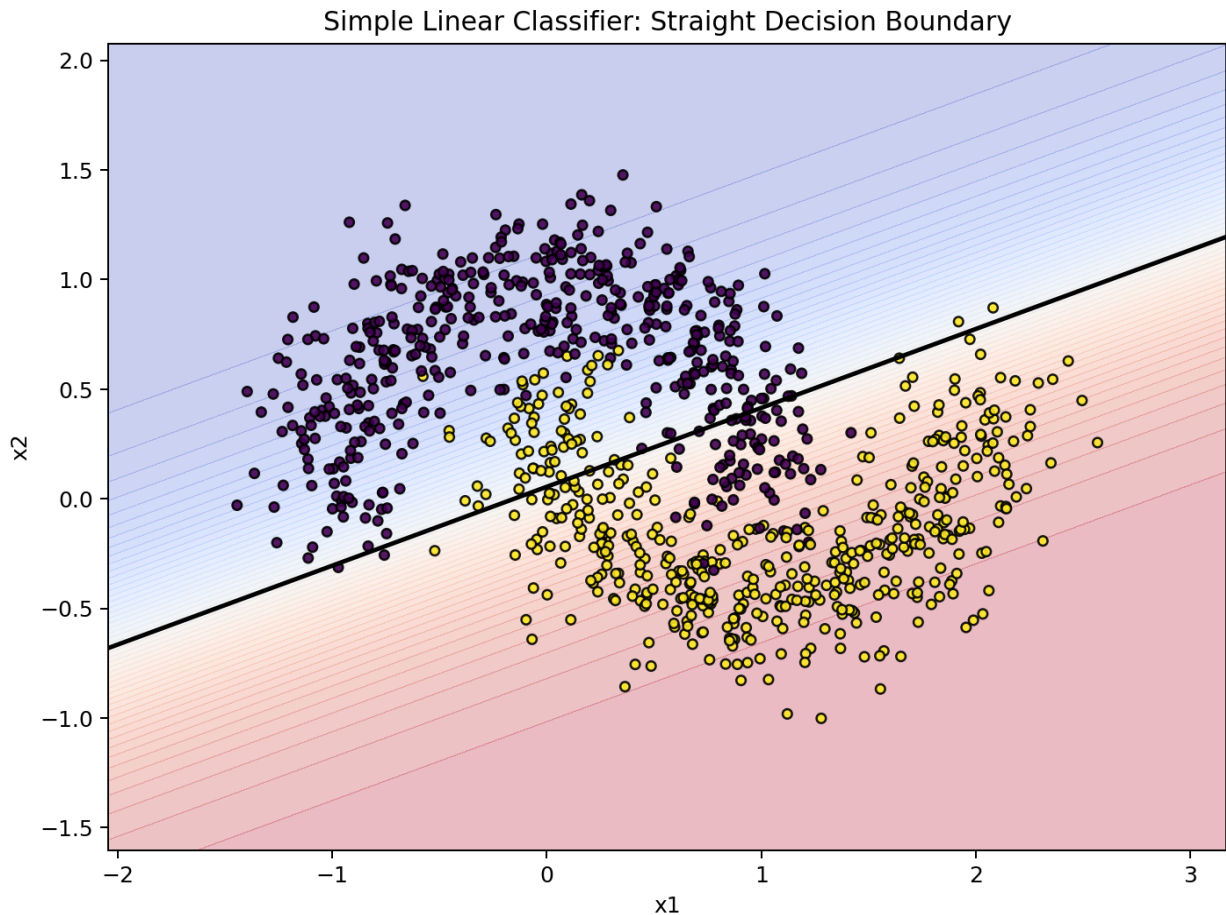
```
predictions = model(X_train)
loss = loss_function(predictions, y_train)
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

In the uploaded notebook, the dataset is generated with 1000 samples, converted to tensors, and split into 800 training examples and 200 test examples.

2. First Attempt: Linear Model

The notebook first creates `nn.Linear(2, 1)`. Because there are two input features, `x1` and `x2`, the model has two weights and one bias. With a sigmoid added, the raw linear output becomes a probability between 0 and 1.

```
model = nn.Sequential(  
    nn.Linear(2, 1),  
    nn.Sigmoid()  
)
```



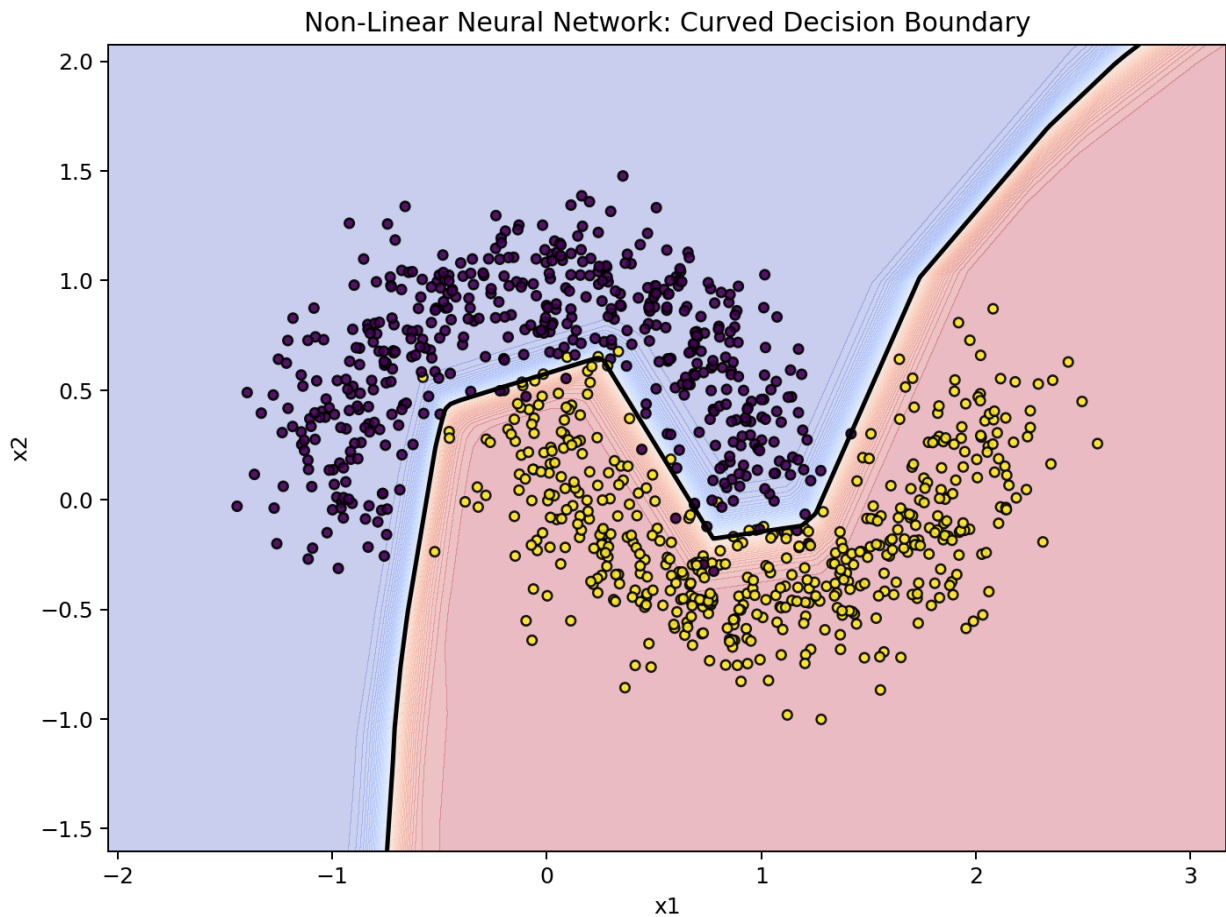
Why it fails: one linear neuron can only create a single straight decision boundary. The two moon-shaped classes bend around each other, so one straight line cannot separate them cleanly.

In the notebook, the first linear classifier uses BCELoss and SGD. After training, the displayed loss levels off around 0.2864. The key lesson is that this is an *architecture limitation*: a straight boundary is too simple for this dataset.

3. Improved Model: Non-Linear Neural Network

The notebook then adds hidden layers and ReLU activations. The final model is:

```
model = nn.Sequential(  
    nn.Linear(2, 16),  
    nn.ReLU(),  
    nn.Linear(16, 16),  
    nn.ReLU(),  
    nn.Linear(16, 1)  
)
```



Why this works: the hidden layers plus ReLU allow the network to build a non-linear decision boundary. Instead of one straight line, the model can bend the boundary around the moon-shaped classes.

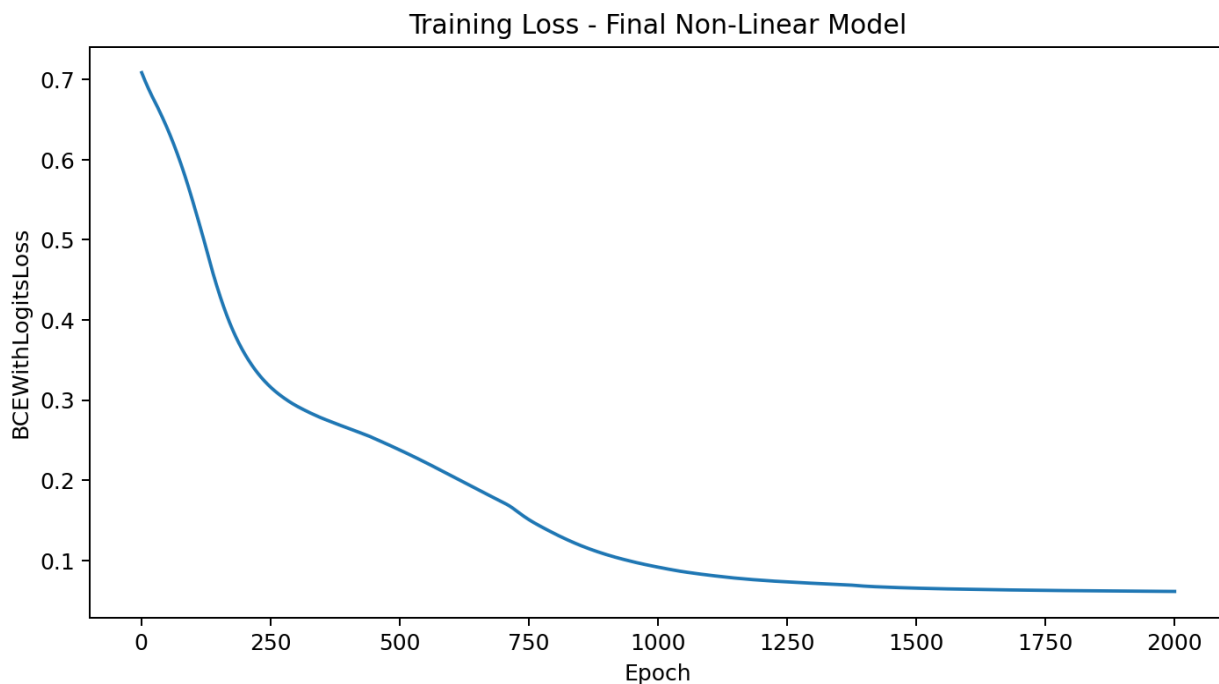
The uploaded notebook trains this model with BCEWithLogitsLoss and Adam for 2000 epochs. The recorded loss falls from 0.2932 at epoch 200 to 0.0613 at epoch 2000, and the recorded test accuracy is 99.0%.

4. What Each PyTorch Line Is Doing

Concept	Code	Meaning
Forward pass	<code>predictions = model(X_train)</code>	Send the inputs through the network and get predictions.
Loss	<code>loss = loss_function(predictions, y_train)</code>	Measure how wrong the predictions are.
Backward	<code>loss.backward()</code>	Compute gradients for the model parameters.
Optimizer step	<code>optimizer.step()</code>	Use the gradients to change weights and biases.
Zero gradients	<code>optimizer.zero_grad()</code>	Clear old gradients before the next iteration.
Evaluation	<code>model.eval() + torch.no_grad()</code>	Switch to evaluation behavior and avoid gradient tracking when only predicting.

Important distinction: `loss.backward()` calculates the gradients; it does not update the weights. `optimizer.step()` performs the update.

5. Training Results and What to Look For



Model	Architecture	Notebook result
Linear baseline	$2 \rightarrow 1 \rightarrow \text{Sigmoid}$	Loss shown around 0.2864 after 1000 epochs
Non-linear (intermediate)	$2 \rightarrow 8 \rightarrow \text{ReLU} \rightarrow 1 \rightarrow \text{Sigmoid}$	Loss shown as 0.2446 after 1000 epochs
Final non-linear	$2 \rightarrow 16 \rightarrow \text{ReLU} \rightarrow 16 \rightarrow \text{ReLU} \rightarrow 1$	Loss 0.0613 at epoch 2000; test accuracy 99.0%

The main learning outcome is not the final percentage. It is understanding **why** the architecture changes the decision boundary and how the same forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update loop trains both models.

6. Clean Practice Version

```
import torch
import torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

# 1. Dataset
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# 2. Tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# 3. Train / test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 4. Model
model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 16),
    nn.ReLU(),
    nn.Linear(16, 1)
)

# 5. Loss + optimizer
loss_function = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# 6. Training
for epoch in range(2000):
    logits = model(X_train)
    loss = loss_function(logits, y_train)

    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

# 7. Test
model.eval()
with torch.no_grad():
    logits = model(X_test)
    probabilities = torch.sigmoid(logits)
    predictions = (probabilities >= 0.5).float()

accuracy = (predictions == y_test).float().mean()
print('Test Accuracy:', accuracy.item() * 100, '%')
```

This clean version follows the same workflow as the uploaded notebook, while presenting the final, stronger classifier in one place.

7. PyTorch Classification Cheat Sheet

- **Tensor:** PyTorch data structure used for model calculations.
- **Model:** learnable function containing weights and biases.
- **Forward pass:** input goes through the model to produce a prediction.
- **Loss:** numerical measure of prediction error.
- **Gradient:** tells how a parameter should move to reduce loss.
- **Backpropagation:** computes those gradients.
- **Optimizer:** uses gradients to update parameters.
- **Epoch:** one full pass of the training data through the training process.
- **Classification threshold:** here, probability ≥ 0.5 becomes class 1.

Practice next: change $16 \rightarrow 8$, $16 \rightarrow 32$, add/remove a hidden layer, change ReLU to Tanh, change Adam to SGD, and watch the decision boundary. That is the model-building practice that will make PyTorch feel natural.